

Les Hooks de React

Séance 8 - useState, useEffect, useContext + CORS

Cours 420-941-MA - Collège de Maisonneuve - Groupe 25604

règles des hooks - dépendances - nettoyage - contexte - CORS - Pokédex full-stack

Où on en est, où on va

Récap S7 et objectifs S8

Acquis (S7)

- App React avec Vite
- JSX, composants, props
- useState (state local)
- Listes (.map + key), formulaires

Objectifs de la séance

- Les RÈGLES des hooks
- useEffect en profondeur (dépendances, nettoyage)
- useContext : un état global sans prop drilling
- Comprendre et régler CORS
- EXERCICE : brancher le Pokédex (back) à un front React

L'objectif du jour. Les hooks sont le vrai super-pouvoir de React : ils donnent de la MEMOIRE et du CYCLE DE VIE aux composants. On maîtrise les 3 plus utilisés, on comprend enfin l'erreur CORS, puis on relie notre backend Pokédex à une interface React que l'on construit ensemble. Le full-stack, pour de vrai.

PARTIE 1

Les hooks : les règles du jeu

Un hook, c'est quoi ?

Une fonction qui 's'accroche' à React

Un hook est une fonction spéciale (son nom commence toujours par 'use') qui permet à un composant d'utiliser des fonctionnalités de React : garder un état (useState), réagir à des événements du cycle de vie (useEffect), lire un contexte (useContext)...

Règle 1 : au sommet

- Appeler les hooks tout EN HAUT du composant
 - Jamais dans un if, une boucle, ou une fonction imbriquée
- > l'ordre doit rester stable

Règle 2 : où ?

- Uniquement dans un COMPOSANT
 - React (ou un hook personnalisé)
 - Pas dans une fonction ordinaire,
 - pas dans un événement
- > React suit vos hooks par leur ordre

Pourquoi ces règles ? React identifie chaque hook par SA POSITION dans le composant (1er, 2e, 3e...). Si l'ordre change d'un rendu à l'autre (à cause d'un if), React s'emmêle. D'où : toujours le même nombre de hooks, dans le même ordre.

PARTIE 2

useState & useEffect

useState : rappel express

La mémoire d'un composant

```
const [valeur, setValeur] = useState(initiale)

const [compte, setCompte] = useState(0)
const [nom, setNom] = useState('')
const [liste, setListe] = useState<any[]>([])

// on NE modifie jamais 'valeur' directement
setCompte(compte + 1) // -> re-rend le composant
```

TSX

3 idées

- [valeur, setter]
- on lit 'valeur',
• on change via le setter
- changer l'état
• = re-rendu automatique

useState répond à la question « qu'est-ce qui change et doit rester en mémoire ? ». useEffect, lui, répond à « quoi FAIRE quand quelque chose change ? » (charger des données, démarrer un minuteur...). Les deux travaillent souvent ensemble.

useEffect : l'anatomie

Un effet + un tableau de dépendances

```
useEffect(() => {  
  // L'EFFET : le code à exécuter  
  console.log('le composant a change')  
  
}, [dependances])  
// ^^^^^^^^^ QUAND relancer l'effet
```

TSX

À quoi ça sert

- Exécuter du code APRES
- le rendu (affichage)
- Idéal pour : appels API,
- minuteurs, abonnements,
- synchro avec l'extérieur

Deux parties :

- **L'effet (la fonction)** : le CODE à exécuter (ex. axios.get...)
- **Le tableau de dépendances []** : QUAND relancer cet effet (c'est le point clé, slide suivant)

Le tableau de dépendances

Trois cas à connaître par cœur

Écriture	Quand l'effet se lance	Usage typique
<code>}, [])</code>	une seule fois (au montage)	charger des données au démarrage
<code>}, [x])</code>	au montage ET quand x change	réagir à un changement (filtre, id)
<code>})</code> (rien)	après CHAQUE rendu	rare - souvent une erreur / boucle

```
// s'exécute UNE FOIS au démarrage
useEffect(() => { chargerPokemons() }, [])

// se relance CHAQUE FOIS que 'recherche' change
useEffect(() => { filtrer(recherche) }, [recherche])
```

TSX

Le nettoyage (cleanup)

Ranger derrière soi : minuteurs, abonnements

```
useEffect(() => {  
  const id = setInterval(() => {  
    console.log('tic')  
  }, 1000)  
  
  // fonction de NETTOYAGE (optionnelle)  
  return () => clearInterval(id)  
}, [])
```

TSX

La fonction retournée

- Si l'effet 'ouvre' quelque chose (minuteur, abonnement, connexion), il faut le 'refermer'
- React appelle ce return quand le composant disparaît (ou avant de relancer l'effet)

Sans nettoyage : le minuteur continue même après la disparition du composant -> fuites de mémoire et bugs. Pour un simple `axios.get`, pas besoin de nettoyage ; il devient essentiel pour les minuteurs et les abonnements.

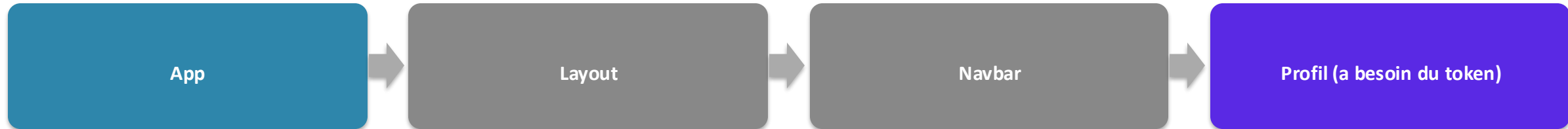
PARTIE 3

useContext : le 3e hook à connaître

Le problème : le 'prop drilling'

Faire descendre une donnée sur 4 étages

Certaines données sont utiles PARTOUT dans l'app : l'utilisateur connecté, le token, le thème... Les faire passer de props en props à travers tous les composants intermédiaires est pénible et fragile. C'est le 'prop drilling'.



token passé de props en props alors que seuls App et Profil s'en servent...

useContext résout ça : on met la donnée dans un 'contexte' accessible directement par n'importe quel composant, sans la faire transiter par tous les intermédiaires.

useContext en 3 temps

Créer -> Fournir -> Consommer

```
// 1) CREER le contexte (authContext.tsx)
import { createContext } from 'react'
export const AuthContext = createContext<string | null>(null)

// 2) FOURNIR la valeur (App.tsx), en haut de l'arbre
const [token, setToken] = useState<string | null>(null)
return (
  <AuthContext.Provider value={token}>
    <Layout />
  </AuthContext.Provider>
)

// 3) CONSOMMER (n'importe où, ex. Profil.tsx)
const token = useContext(AuthContext) // directement !
```

TSX

Le trio

- createContext :
- on déclare la 'boîte'
-
- Provider :
- on remplit la boîte,
- en HAUT de l'arbre
-
- useContext :
- on ouvre la boîte,
- depuis n'importe quel
- composant en dessous

useContext : quand (et quand pas)

Un outil pour l'état PARTAGÉ

Bon usage

- L'utilisateur / le token (auth)
- Le thème (clair / sombre)
- La langue
- Un panier partagé
- -> des données GLOBALES, stables

À éviter

- Tout mettre dans un contexte
- Des données très locales
- Remplacer chaque prop
- -> garde useState pour le local,
- le contexte pour le vraiment partagé

Règle simple : props pour ce qui est local (parent -> enfant direct), contexte pour ce dont BEAUCOUP de composants ont besoin. Dans notre projet, le token JWT est le candidat parfait pour un contexte.

PARTIE 4

CORS : le mur entre front et back

CORS : c'est quoi cette erreur ?

Cross-Origin Resource Sharing

Votre front tourne sur localhost:5173, votre backend sur localhost:3000. Ce sont deux ORIGINES différentes. Par sécurité, le navigateur BLOQUE par défaut les requêtes d'une origine vers une autre. Résultat : une erreur 'CORS policy' dans la console.



IMPORTANT : ce n'est pas un bug de votre code ni un problème de React. C'est une protection du NAVIGATEUR. Et surtout : elle se règle CÔTÉ BACKEND, car c'est le serveur qui décide qui a le droit de l'appeler.

La solution : cors côté backend

Quelques lignes dans Express

1. Installer (dans le dossier du BACKEND) :

```
npm install cors    &&    npm install -D @types/cors
```

bash

2. Activer dans src/server.ts (AVANT les routes) :

```
import cors from 'cors'

// simple : autoriser tout le monde (OK en developpement)
app.use(cors())

// plus strict : n'autoriser QUE notre front
app.use(cors({ origin: 'http://localhost:5173' })))
```

TS

En développement, app.use(cors()) suffit. En production, on restreint aux origines de confiance (le domaine de votre vrai front).

PARTIE 5

Exercice : le Pokédex full-stack

Exercice : brancher le Pokédex

Votre backend rencontre son interface

On construit un petit front React qui consomme VOTRE backend Pokédex (celui des séances précédentes). Les routes réelles :

Méthode	Route	Ce que ça fait
GET	/pokedex/api	liste de Pokemon (depuis la PokeAPI)
GET	/pokedex	les Pokemon capturés (base de données)
POST	/pokedex/capturer/:nom	capturer un Pokemon

Côté backend (1 fois)

- Installer et activer cors
- `app.use(cors())`
- Relancer le serveur (port 3000)

Côté frontend (React)

- Créer l'app Vite + axios
- `useEffect` + axios -> afficher la liste
- Un bouton 'Capturer' -> POST
- Gérer chargement / erreur

Aperçu : le cœur du front

useState + useEffect + axios

TSX

```
const [pokemons, setPokemons] = useState<any[]>([])
const [chargement, setChargement] = useState(true)

useEffect(() => {
  axios.get('http://localhost:3000/pokedex/api')
    .then((res) => setPokemons(res.data))
    .catch(() => setErreur('Echec du chargement'))
    .finally(() => setChargement(false))
}, [])

async function capturer(nom: string) {
  await axios.post(`http://localhost:3000/pokedex/capturer/${nom}`)
}

// dans le JSX : pokemons.map((p) => <li key={p.name}>{p.name}
//   <button onClick={() => capturer(p.name)}>Capturer</button></li>)
```

Le détail pas à pas est dans le guide : EXERCICE-POKEDEX-FRONT.md

Récapitulatif de la séance

Ce qu'il faut retenir

Les hooks

- Règles : au sommet,
- dans un composant
- useState : mémoire
- useEffect : cycle de vie
- useContext : global

useEffect

- effet + dépendances
- [] : une fois
- [x] : quand x change
- return : nettoyage
- (minuteurs, abonnements)

CORS & projet

- erreur du navigateur
- réglée côté backend
- npm install cors
- app.use(cors())
- Pokédex full-stack

À mémoriser :

```
useEffect(() => { /* charger */ }, []) // au démarrage
const token = useContext(AuthContext) // etat global
app.use(cors())                       // cote backend
```

mix

Pour la prochaine séance

On consolide et on prépare la suite

À faire de votre côté

- Activer cors dans le backend Pokédex
- Créer le front React (Vite)
- Afficher /pokedex/api (useEffect)
- Bouton 'Capturer' (POST)
- Afficher /pokedex (les captures)
- Gérer chargement / erreur

Au programme ensuite

- Le routing (plusieurs pages)
- Auth complète (contexte + login)
- Formulaires avancés
- Composants réutilisables + CSS
- Vers Next.js (full-stack)
- Déploiement (Vercel)

Votre Pokedex a maintenant un BACKEND et un FRONTEND qui se parlent.

Hooks + axios + CORS : vous tenez toute la chaine full-stack.



Questions ?

Prochaine étape : brancher le front sur votre Pokédex.

`useState, useEffect, useContext...` et cors. En avant !